Single-source shortest paths: exercises

24-E01 In-place and synchronous states differ

On vertices(s, a, b, c, t, z), run in-place Bellman–Ford with ordered edges ct3, at 8, ac1, bt12, bc5, ba−2, sa4, sb1. List every round's distances, parent changes, and first unchanged round. Reorder edges to sa, sb, ba, ac, ct, at, bc, bt. What is the first-round state? Can the synchronous edge-count DP obtain it in one round? Explain each difference.

24-E02 Negative-cycle certificates and affected vertices

Add ca:−2 to 24-E01, and separate vertices p, q with pq:−4, qp:1. Classify all eight source distances as finite,∞, or−∞. Give a reachable negative cycle and a formula for a walk to t after r repetitions. Why must the pq cycle not directly mark p, q as−∞? Prove that forward search from synchronous layer-n seeds gives exactly the affected set.

24-E03 Shortest paths and critical paths

Why does 24-E01 permit DAG shortest paths? Give a topological order and all updates after each scanned vertex. For jobs A:3, B:2, C:4, D:1 with A→C, B→C, C→D, find earliest starts and finishes, a critical path, and total work. Can one processor attain the critical-path finish bound?

24-E04 Dijkstra's no-reopening contract

Change ba in 24-E01 to+2. List every extraction, distance, and parent from s. Why must the algorithm continue after first discovering t? For sa2, sb5, ba−4, at 2, trace the failure of the standard no-reopening version and identify the exact invalid inequality in its proof. Does the counterexample contain a negative cycle?

24-E05 Two certificates for difference constraints

For $x_2-x_1\le-2$, $x_3-x_2\le-1$, $x_1-x_3\le4$, construct arcs, run all-zero Bellman–Ford, verify every constraint, and normalize min $x_i=0$. Change the last constant to 2 and give an algebraic infeasibility certificate. Represent the additional equality $x_1-x_3=3$ by two arcs. If the original constants become−1.2,−0.2,2.8 and every variable must be integer, is the system feasible?

Single-source shortest paths: exercises (continued)

24-E06 Zero-weight predecessors and independent verification

For sa0, ab0, ba0, proposed distances are all 0 with parent[a]=b, parent[b]=a. Why do tight parent edges not give a shortest-path tree? Give legal parents. Prove which inequality direction follows from a tight rooted tree and which from all input-edge inequalities. For each part, give a wrong output accepted if the other part is omitted.

24-E07 Reconstructing a cycle from the edge-count DP

For sa1, ab−3, ba1, compute synchronous D_0 through D_3. Choose a strictly improved layer 3 entry, reconstruct its exactly-three-edge walk, and extract a negative cycle. Prove generally why a layer-n improvement requires exactly n edges. Why does merely containing a cycle not establish negativity? Compare full-history and two-layer space.

24-E08 Algorithm choices and necessary cost assumptions

Choose algorithms and output semantics for(a) directed unit weights,(b) a negative-weight DAG,(c) a sparse nonnegative graph,(d) a graph with a reachable negative cycle. For $n=10^4$, $m=3\times10^4$, give leading cost terms for Bellman–Ford, indexed binary Dijkstra, Fibonacci Dijkstra, and linear-key scanning. For m=0, why can neither Bellman–Ford nor difference-constraint solving claim zero time?

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study